

CSCE 611 OPERATING SYSTEMS · MODULE 10 · LECTURE NOTES

I/O and Device Drivers

Talking to hardware that is slow, asynchronous, and hostile.

MODULE OUTCOMES

- Explain memory-mapped I/O and port I/O.
- Compare polling and interrupts and say when each is right.
- Explain DMA and why it matters.
- Explain the top-half/bottom-half split in interrupt handling.
- Explain the block layer and I/O scheduling.

1 Reaching a device

	Memory-mapped I/O	Port I/O
Mechanism	Device registers are assigned physical addresses; the CPU reads and writes them with ordinary load and store instructions.	A separate address space accessed with dedicated instructions (in/out on x86).
Requirement	The mapping must be marked uncacheable .	No cache interaction by construction.
Status	Universal on modern architectures.	Legacy, x86-only, retained for a few old devices.

DEVICE REGISTERS VIOLATE EVERY ASSUMPTION ABOUT MEMORY

Reads have side effects. Reading a status register may clear a pending interrupt or pop an entry from a hardware FIFO. Reading twice is not the same as reading once.

Values change without being written. A status register changes because the device changed it.

Writes are not idempotent. Writing a command register twice issues the command twice.

So the compiler must not cache the value in a register, must not reorder accesses, and must not eliminate reads it believes are redundant. **This is what volatile is for** — memory-mapped hardware, not thread synchronisation, which it does not provide (CSCE 614 Module 12).

And volatile constrains only the compiler. The processor may still reorder, so explicit memory barriers are required as well, and the mapping must be uncacheable or the cache will return stale values and absorb writes.

2 Knowing when the device is ready

	Polling	Interrupts
How	Read a status register in a loop until it indicates ready.	The device asserts a signal; the CPU traps to a handler.
CPU cost while waiting	A full core.	None.
Latency	Minimal — noticed almost immediately.	Interrupt entry, context save, handler dispatch — microseconds.
Best for	Devices that respond in less time than an interrupt costs; very high event rates.	Slow devices and infrequent events.
Examples	NVMe at high queue depth; a NIC under heavy load.	Keyboard, mouse, spinning disk completion.

The historical direction was from polling to interrupts, as devices became slower relative to processors. Fast storage and high-speed networking have partly reversed it: when a device responds in less time than an interrupt costs to deliver, polling is simply cheaper.

INTERRUPT LIVELOCK

A network interface receiving ten million packets per second will generate ten million interrupts per second if each packet raises one. Every interrupt costs a context save, a handler entry, and cache and branch-predictor disturbance (CSCE 614 Modules 04 and 05).

The system spends all of its time entering and leaving interrupt handlers and never reaches the code that would actually process packets. Throughput collapses toward zero while CPU utilisation sits at 100% — a livelock in the sense of Module 05.

The standard fix is to switch modes adaptively: on the first interrupt, **disable** that device's interrupts and poll the queue; when the queue drains, re-enable them and go back to sleep.

Linux's **NAPI** is exactly this, and it gives interrupt-driven efficiency at low rates with polled efficiency at high rates, without the administrator choosing.

3 DMA

With **programmed I/O**, the CPU moves every byte between the device and memory. For a device capable of 1 GB/s this consumes an entire core doing nothing but copying — which is why it does not scale.

Direct memory access instead hands the device a physical buffer address and a length. The device transfers data to or from memory itself and raises an interrupt on completion. The CPU is free throughout.

DMA CREATES A CACHE COHERENCE OBLIGATION

The device writes physical memory directly, bypassing the CPU's caches. Those caches do not know.

Before a device-read transfer (CPU → device), the buffer must be flushed from cache, or the device may read stale data from memory while the current data sits dirty in cache.

After a device-write transfer (device → CPU), the buffer must be invalidated, or the CPU may read a stale cached copy instead of the freshly transferred data.

Many systems provide *coherent* DMA, where hardware snoops and handles this automatically. Many embedded systems do not, and omitting the flush or invalidate produces intermittent corruption that depends on cache state — among the least pleasant bugs in systems programming.

An **IOMMU** adds address translation and protection for device accesses, which contains a buggy or malicious device and is what makes it safe to assign a device directly to a virtual machine.

4 Interrupt handlers

An interrupt handler runs with interrupts disabled, preempting whatever was executing. While it runs, *no other interrupt can be serviced* — including the timer.

WHY A LONG HANDLER IS DANGEROUS

Blocking all interrupts delays every other device, raising latency and potentially overflowing hardware buffers.

Worse, it delays the **timer** interrupt. Miss enough timer ticks and the system clock drifts and scheduling decisions are deferred, which affects everything.

An interrupt handler also **cannot sleep**: there is no process context to block, and blocking with interrupts disabled would deadlock the machine.

```
// TOP HALF -- interrupts disabled. Minimal work only.
void disk_intr(void) {
    ack_device();           // acknowledge, so it stops asserting
    struct buf *b = dequeue_completed();
    b->flags |= B_VALID;
    schedule_bottom_half(b); // defer the rest
}

// BOTTOM HALF -- runs later, interrupts enabled, may sleep.
void disk_bh(struct buf *b) {
    wakeup(b);
    update_statistics();
    maybe_issue_next_request();
}
```

The top half does only what must happen immediately: acknowledge the device so it stops asserting the interrupt, capture any state that would otherwise be lost, and schedule the remainder. Everything else runs in a **bottom half** with interrupts enabled.

Linux provides several deferral mechanisms with different properties: *softirqs* (fixed set, high performance), *tasklets* (dynamically created, serialised per tasklet), and *workqueues* (run in process context and therefore may sleep). The choice depends on whether the deferred work needs to block.

5 The block layer

Requests do not go straight to the device. The kernel queues them, and before dispatch it can merge and reorder.

- **Merging** combines adjacent requests into one larger transfer. This is a substantial win on every device type, because per-request overhead is significant and large transfers amortise it.

- **Reordering** was historically the larger win. On a spinning disk, seek time dominates, and servicing requests in block-address order — the *elevator* algorithm, sweeping the head across the platter — could improve throughput several-fold.
- **On an SSD there is no seek.** Access time is essentially independent of address, so elevator ordering buys almost nothing and the complexity is wasted.

A CASE WHERE THE RIGHT ANSWER REVERSED

Decades of I/O scheduler research optimised for seek time. When storage stopped having a seek, most of that work became irrelevant — and some of it became actively harmful, because the scheduler introduced latency to perform reordering that no longer helped.

Modern NVMe devices expose many deep queues, typically one per CPU core, and perform their own internal scheduling with far better knowledge of the flash layout than the kernel has. For these, Linux's none scheduler — do nothing, submit immediately — is frequently the best choice.

It is a useful reminder that systems knowledge has a half-life, and that an optimisation is a claim about hardware that may stop being true.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
OSTEP — Chapters 36–37 (I/O Devices, Hard Disk Drives)	Polling, interrupts, DMA, and disk scheduling with the seek-time analysis.
xv6 book — Chapter 5 (Interrupts and device drivers)	A complete disk and console driver, short enough to read entirely.
Linux Device Drivers, 3rd edition (free)	Dated in specifics and still the best free explanation of the concepts — memory-mapped I/O, DMA, deferred work.
Mogul & Ramakrishnan — Eliminating Receive Livelock (free)	The paper behind NAPI. Clear, and the problem it describes recurs wherever event rates grow.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Read xv6's console and disk drivers. Identify the memory-mapped register accesses and the interrupt handler, and classify the work as top-half or bottom-half.

- 2 Remove volatile from a device register access and inspect the generated assembly at -O2. Confirm the compiler hoists the read out of the polling loop.
- 3 Implement polling and interrupt-driven versions of a device driver in xv6. Measure CPU utilisation and latency for each at low and high event rates.
- 4 Simulate interrupt livelock: generate events faster than the handler can retire them and observe throughput collapsing while the CPU saturates. Then implement NAPI-style adaptive polling and measure again.
- 5 Split an interrupt handler into top and bottom halves. Measure the maximum interrupt-disabled duration before and after.
- 6 Measure the effect of request merging: issue a thousand sequential 4 KB reads and then one 4 MB read, and compare total time.
- 7 Compare I/O schedulers on a spinning disk and an SSD, if you have access to both, with a random-access workload. Explain the difference.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why must memory-mapped device registers be uncacheable, and why is volatile necessary but insufficient?
- 2 Give the trade-off between polling and interrupts, and explain why fast devices have swung it back toward polling.
- 3 What is interrupt livelock, and how does NAPI address it?
- 4 What does DMA buy, and what obligation does it create for the driver?
- 5 Why must an interrupt handler be short, and why can it not sleep?
- 6 What goes in the top half and what in the bottom half?
- 7 Why did I/O scheduling matter so much for spinning disks and so little for NVMe?